

# Fenix FEM

Plataforma de Análisis por Elementos Finitos en Python

---

## Manual de Usuario y Guía de Referencia

Documentación Oficial y Arquitectura del Sistema

Versión 2.0

**Autor:** Jaime Retama Velasco

Facultad de Estudios Superiores Aragón  
Universidad Nacional Autónoma de México

13 de mayo de 2026

# Índice general

|                                                                                                                       |            |
|-----------------------------------------------------------------------------------------------------------------------|------------|
| <b>Resumen Ejecutivo</b>                                                                                              | <b>III</b> |
| <b>1. Introducción y Configuración del Entorno</b>                                                                    | <b>1</b>   |
| 1.1. Filosofía del Proyecto                                                                                           | 1          |
| 1.1.1. Modelo <i>Data-Driven</i>                                                                                      | 1          |
| 1.2. Requisitos del Sistema                                                                                           | 1          |
| 1.2.1. Dependencias                                                                                                   | 1          |
| 1.3. Herramientas Externas Recomendadas                                                                               | 2          |
| 1.4. Ejecución de un Análisis                                                                                         | 2          |
| <b>2. Arquitectura del Programa</b>                                                                                   | <b>3</b>   |
| 2.1. Separación Elemento $\leftrightarrow$ Material                                                                   | 3          |
| 2.2. Auto-Registro de Componentes                                                                                     | 3          |
| 2.3. Validación Temprana del Input                                                                                    | 4          |
| <b>3. Archivo de Entrada YAML</b>                                                                                     | <b>5</b>   |
| 3.1. Geometría: <code>nodes</code> y <code>elements</code>                                                            | 5          |
| 3.2. Importación de Malla Gmsh: <code>mesh</code>                                                                     | 6          |
| 3.2.1. Mapeo Avanzado: <code>mesh_physical_groups</code>                                                              | 6          |
| 3.3. Materiales                                                                                                       | 6          |
| 3.4. Condiciones de Frontera                                                                                          | 7          |
| 3.4.1. Por nodo: <code>boundary_conditions_by_node</code>                                                             | 7          |
| 3.4.2. Por coordenada: <code>boundary_conditions_by_coord</code>                                                      | 7          |
| 3.4.3. Por grupo físico: <code>boundary_conditions_by_group</code>                                                    | 7          |
| 3.4.4. Restricciones multipunto (MPC): <code>linear_constraints</code>                                                | 7          |
| 3.5. Cargas Puntuales                                                                                                 | 8          |
| 3.6. Solver                                                                                                           | 8          |
| 3.6.1. Bloque <code>convergence</code>                                                                                | 9          |
| 3.7. Salida: <code>output</code>                                                                                      | 10         |
| <b>4. Catálogo de Elementos Finitos</b>                                                                               | <b>11</b>  |
| 4.1. Elementos 1D                                                                                                     | 11         |
| 4.1.1. Armaduras: <code>Truss2D</code> / <code>Truss2DCorot</code> / <code>Truss3D</code> / <code>Truss3DCorot</code> | 11         |
| 4.1.2. Cables: <code>Cable2DCorot</code> / <code>Cable3DCorot</code>                                                  | 12         |
| 4.1.3. Marcos 2D: <code>Frame2DEuler</code> / <code>Frame2DTimoshenko</code> / <code>Frame2DEulerCorot</code>         | 12         |

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| 4.1.4. Marco 3D: <b>Frame3D</b> . . . . .                                                  | 13        |
| 4.2. Elementos 2D . . . . .                                                                | 14        |
| 4.2.1. Cuadrilátero Bilineal: <b>Quad4</b> . . . . .                                       | 14        |
| 4.2.2. Triángulo de Deformación Constante: <b>Tri3</b> . . . . .                           | 14        |
| <b>5. Catálogo de Modelos Constitutivos</b>                                                | <b>15</b> |
| 5.1. Elasticidad Lineal . . . . .                                                          | 15        |
| 5.1.1. <b>Elastic1D</b> — elasticidad lineal axial . . . . .                               | 15        |
| 5.1.2. <b>Elastic2D</b> — elasticidad lineal isótropa 2D . . . . .                         | 15        |
| 5.2. Plasticidad . . . . .                                                                 | 15        |
| 5.2.1. <b>Elastoplastic1D</b> — plasticidad J2 axial con endurecimiento isótropo . . . . . | 15        |
| 5.2.2. <b>VonMises2D</b> — plasticidad J2 plana con endurecimiento isótropo . . . . .      | 16        |
| 5.3. Daño Mecánico . . . . .                                                               | 16        |
| 5.3.1. <b>IsotropicDamage1D</b> — daño isótropo 1D con softening exponencial . . . . .     | 16        |
| 5.3.2. <b>IsotropicDamage2D</b> — daño isótropo 2D . . . . .                               | 17        |
| 5.4. Elasticidad Unilateral (Cable) . . . . .                                              | 17        |
| 5.4.1. <b>CableMaterial1D</b> — cable lineal en tensión, nulo en compresión . . . . .      | 17        |
| <b>6. Esquemas de Solución</b>                                                             | <b>19</b> |
| 6.1. <b>LinearSolver</b> — solución directa lineal . . . . .                               | 19        |
| 6.2. <b>NonlinearSolver</b> — Newton-Raphson incremental con paso adaptativo . . . . .     | 19        |
| 6.3. <b>ArcLengthSolver</b> — longitud de arco cilíndrico (Crisfield) . . . . .            | 20        |
| <b>7. Post-Procesamiento y Visualización</b>                                               | <b>21</b> |
| 7.1. Formato VTU (VTK XML) . . . . .                                                       | 21        |
| 7.1.1. Datos Nodales ( <i>Point Data</i> ) . . . . .                                       | 21        |
| 7.1.2. Datos de Elemento ( <i>Cell Data</i> ) . . . . .                                    | 21        |
| 7.2. Animación con Archivo PVD . . . . .                                                   | 21        |
| 7.3. Salida en Texto Plano (Estilo FEAP) . . . . .                                         | 21        |
| 7.4. Workflow ParaView . . . . .                                                           | 22        |
| <b>8. Ejemplos Completos</b>                                                               | <b>23</b> |
| 8.1. Marco 2D — Viga en Voladizo . . . . .                                                 | 23        |
| 8.2. Placa Continua con Plasticidad J2 (Gmsh) . . . . .                                    | 24        |
| 8.3. Cable Pretensado en Régimen Corotacional . . . . .                                    | 25        |
| <b>9. Uso Avanzado: API Programática</b>                                                   | <b>27</b> |
| 9.1. Patrón Fachada . . . . .                                                              | 27        |
| <b>10. Diagnóstico de Problemas</b>                                                        | <b>29</b> |
| <b>Índice Alfabético</b>                                                                   | <b>30</b> |

# Resumen Ejecutivo

**Fenix FEM** es un programa de elementos finitos en aproximación de desplazamientos, escrito en Python, dirigido a la investigación en mecánica de sólidos y al análisis estructural. Cubre problemas mecánicos en régimen lineal y no lineal — tanto material (plasticidad, daño, elasticidad unilateral) como geométrico (formulaciones corotacionales) — y está pensado para crecer en el tiempo incorporando nuevos elementos, materiales y métodos de solución sin reescribir el núcleo del programa.

El catálogo actual incluye:

- **10 elementos finitos 1D** en el plano y en el espacio: armaduras, cables y marcos en variantes lineales y corotacionales.
- **2 elementos finitos 2D** continuos: cuadrilátero bilineal **Quad4** y triángulo lineal **Tri3**.
- **7 modelos constitutivos**: elasticidad lineal 1D y 2D, plasticidad J2 1D y 2D, daño isótropo 1D y 2D, y elasticidad unilateral para cables.
- **3 esquemas de solución**: solver lineal directo, Newton-Raphson incremental con paso adaptativo, y longitud de arco cilíndrico de Crisfield para snap-through y snap-back.

La interacción del usuario con el motor se realiza mediante un único **archivo de entrada YAML** que describe geometría, materiales, condiciones de frontera, cargas y configuración del solver. La salida se exporta en formato `.vtu` (VTK XML) para post-procesamiento en *ParaView*, y opcionalmente en texto plano estilo FEAP.

Este manual documenta el uso del programa — sintaxis del archivo YAML, catálogos de componentes y workflow de ejecución. Para detalles internos sobre arquitectura, formulaciones físicas y workflow de extensión por desarrollo asistido por IA, consulte la documentación interna del repositorio (`docs/`, `Reglas.md`).

# Capítulo 1

## Introducción y Configuración del Entorno

### 1.1 Filosofía del Proyecto

Fenix FEM nace como herramienta de investigación en mecánica del medio continuo y análisis no lineal de estructuras. Su diseño persigue dos objetivos simultáneos:

1. **Rigor numérico y físico.** Cada formulación incorporada está validada contra solución analítica o benchmark establecido. La separación estricta **Elemento**  $\leftrightarrow$  **Material** permite combinar cualquier cinemática con cualquier ley constitutiva siempre que sus protocolos sean compatibles.
2. **Crecimiento sin fricción.** La arquitectura está optimizada para que añadir un elemento, material o solver nuevo sea barato. Auto-registro vía decoradores, contratos declarativos (**STRAIN\_DIM**, **DOF\_NAMES**) y validación temprana del input son los mecanismos centrales.

#### 1.1.1 Modelo *Data-Driven*

Toda la orquestación de un análisis se concentra en un archivo `.yaml` legible por humanos. El usuario no necesita escribir código Python para definir un modelo; la API programática queda reservada para casos avanzados (optimización paramétrica, mallas generadas proceduralmente, acoplamiento con otros sistemas).

### 1.2 Requisitos del Sistema

Fenix FEM se ejecuta sobre Python 3.8 o superior en Windows, Linux y macOS sin compilación nativa.

#### 1.2.1 Dependencias

```
pip install numpy scipy pyyaml meshio numba
```

- **numpy** / **scipy**: álgebra lineal densa y dispersa; **spsolve** sobre matrices CSR.
- **pyyaml**: análisis sintáctico del archivo de entrada.
- **meshio**: importación de mallas Gmsh y exportación a VTK.
- **numba**: compilación JIT de los kernels críticos en elementos 2D y plasticidad.

## 1.3 Herramientas Externas Recomendadas

- **Gmsh**: generador de mallas; produce archivos `.msh` consumibles directamente desde el YAML.
- **ParaView**: visualización interactiva de los archivos `.vtu` generados.

## 1.4 Ejecución de un Análisis

El runner principal vive en `examples/ejecutar_yaml.py`. Para ejecutar un modelo:

```
python examples/ejecutar_yaml.py ruta/al/modelo.yaml
```

El script carga el YAML, valida su contenido (acumulando *todos* los errores en una sola pasada antes de abortar), construye el dominio, ensambla el solver pedido, ejecuta el análisis y exporta los resultados según el bloque `output`.

# Capítulo 2

## Arquitectura del Programa

### 2.1 Separación Elemento $\leftrightarrow$ Material

Cada elemento finito declara dos contratos sobre el material que acepta:

- **STRAIN\_DIM**: dimensión Voigt de la deformación que entrega al material. 1 para elementos axiales (truss, cable, marcos — la fibra centroidal); 3 para 2D ( $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ ); 6 para 3D continuo (no implementado todavía).
- **DOF\_NAMES**: lista de grados de libertad por nodo. Por ejemplo, `['ux', 'uy', 'rz']` para un marco 2D.

El protocolo del material es uniforme:

```
sigma, E_tangent, new_state = material.compute_state(epsilon, state_vars)
```

donde `epsilon` tiene dimensión **STRAIN\_DIM**, `sigma` es el esfuerzo en la misma representación, `E_tangent` es la matriz tangente consistente y `state_vars` encapsula las variables internas (deformación plástica, daño acumulado, etc.).

Un material declara igualmente su **STRAIN\_DIM**; el dominio rechaza al construir cualquier emparejamiento incompatible.

### 2.2 Auto-Registro de Componentes

Elementos, materiales y solvers se registran automáticamente al arrancar el programa mediante decoradores:

```
@ElementRegistry.register
class Truss2D(Element):
    DOF_NAMES = ['ux', 'uy']
    STRAIN_DIM = 1
    ...
```

Esto permite que el archivo YAML los referencie por nombre (`type: Truss2D`) sin que el usuario tenga que importar nada manualmente. El módulo `fenix.autodiscover` recorre los paquetes `elements/`, `materials/` y `math/solvers.py` en la primera importación y puebla los registries.

## 2.3 Validación Temprana del Input

El parser YAML acumula todos los errores detectados en una sola pasada (no aborta al primero) y los presenta juntos. Comprueba:

- Ausencia de bloques obligatorios (`nodes`, `materials`, `elements`, `solver`).
- IDs duplicados de nodo, material o elemento.
- Referencias a materiales o nodos inexistentes desde `elements` y `boundary_conditions`.
- Tipos de elemento, material y solver no registrados.
- **Parámetros no aceptados por el constructor del elemento.** Si un elemento define `A` e `I` como parámetros, escribir `large_strains: true` en su entrada YAML produce un error explícito que indica los parámetros válidos para ese tipo.



# Capítulo 3

## Archivo de Entrada YAML

La interacción exclusiva con el motor de Fenix FEM se realiza mediante el archivo de entrada `.yaml`. Un archivo válido contiene los bloques siguientes (algunos opcionales según el caso de uso):

- `nodes` o `mesh` — geometría.
- `materials` — catálogo de leyes constitutivas instanciadas.
- `elements` — conectividad (omisible si se importa malla Gmsh).
- `boundary_conditions_by_node` / `_by_coord` / `_by_group` — restricciones de Dirichlet.
- `point_loads_by_node` / `_by_coord` / `_by_group` — cargas de Neumann puntuales.
- `solver` — tipo de solver y sus parámetros.
- `output` — frecuencia y formato de exportación de resultados.

### 3.1 Geometría: nodes y elements

Para modelos pequeños o estructuras reticulares (armaduras, marcos, cables), la geometría se escribe directamente en el YAML:

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}

elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6
```

Las coordenadas pueden ser de 2 o 3 componentes; los elementos 1D 3D (Truss3D, Cable3DCorot, Frame3D) requieren las tres.

## 3.2 Importación de Malla Gmsh: mesh

Para problemas continuos 2D, se sustituye el bloque `nodes/elements` por una referencia al archivo `.msh`:

```
mesh: "geometria_placa.msh"
mesh_material: 1
mesh_thickness: 0.1
mesh_quadrature: "2x2"
```

| Parámetro       | Descripción                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------|
| mesh            | Ruta al archivo <code>.msh</code> de Gmsh, relativa al YAML.                                    |
| mesh_material   | ID del material por defecto que se asocia a toda la malla.                                      |
| mesh_thickness  | Espesor por defecto (estado plano 2D).                                                          |
| mesh_quadrature | Regla de Gauss: "2x2" (completa, recomendada) o "1x1" (reducida; produce <i>hourglassing</i> ). |

Tabla 3.1: Parámetros del bloque `mesh`.

### 3.2.1 Mapeo Avanzado: mesh\_physical\_groups

Si la geometría tiene zonas con materiales o espesores distintos definidos como *Physical Groups* en Gmsh, se mapean explícitamente:

```
mesh_physical_groups:
  "Zona_Hormigon":
    material: 1
    thickness: 0.20
    quadrature: "2x2"
  "Zona_Acero":
    material: 2
    thickness: 0.05
```

## 3.3 Materiales

Cada material se declara con un `id` único y un `type` registrado en el catálogo:

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
```

```
hypothesis: plane_strain
```

Los parámetros aceptados dependen del material; consulte el Capítulo 5 para el catálogo completo.

## 3.4 Condiciones de Frontera

Fenix FEM ofrece tres mecanismos para imponer restricciones cinemáticas (Dirichlet), elegibles según la facilidad operativa:

### 3.4.1 Por nodo: `boundary_conditions_by_node`

```
boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}
- {node_id: 4, ux: 0.0, uy: 0.0}
```

### 3.4.2 Por coordenada: `boundary_conditions_by_coord`

Escanea todos los nodos y aplica la restricción a aquellos cuya coordenada coincide (dentro de `tol`):

```
boundary_conditions_by_coord:
x_min: {ux: 0.0, uy: 0.0, tol: 1.0e-5}      # empotrar lado izquierdo
rodillos: {coord: 'y', val: 10.0, uy: 0.0, tol: 1.0e-4}
```

Las claves predefinidas `x_min`, `x_max`, `y_min`, `y_max` usan los extremos detectados automáticamente del dominio.

### 3.4.3 Por grupo físico: `boundary_conditions_by_group`

Aplica restricciones a todos los nodos de un *Physical Group* de Gmsh:

```
boundary_conditions_by_group:
"Borde_Empotrado": {ux: 0.0, uy: 0.0}
```

### 3.4.4 Restricciones multipunto (MPC): `linear_constraints`

Para apoyos en plano oblicuo, periodicidad de celda unitaria, uniones rígidas master-slave y simetrías no alineadas con los ejes globales, Fenix FEM admite restricciones afines lineales de la forma  $u_s = g_s + \sum_i \alpha_{si} u_{m_i}$  (ADR 0004 fase 2). Se declaran en el bloque `linear_constraints`:

```
linear_constraints:
# Apoyo a 45 grados en el nodo 4: u_y = u_x.
- slave: {node: 4, dof: uy}
  masters:
    - {node: 4, dof: ux}
  coefficients: [1.0]
```

```
# Periodicidad: u_x del nodo 7 igual al del nodo 3.
- slave: {node: 7, dof: ux}
  masters:
    - {node: 3, dof: ux}
  coefficients: [1.0]

# Union rigida: u_x del satellite N9 sigue al maestro N5 con offset r_y
# =0.5.
- slave: {node: 9, dof: ux}
  masters:
    - {node: 5, dof: ux}
    - {node: 5, dof: rz}
  coefficients: [1.0, -0.5]
g: 0.0
```

Las restricciones se imponen por eliminación directa (no por penalización), de modo que la imposición es exacta a redondeo y preserva la simetría y la positividad definida del sistema. Las cadenas master-slave se resuelven automáticamente por cierre transitivo; los ciclos y las redeclaraciones inconsistentes se detectan en construcción y abortan el análisis con un mensaje específico.

## 3.5 Cargas Puntuales

Misma estructura que las condiciones de frontera, en bloques `point_loads_by_node`, `point_loads_by_coord` y `point_loads_by_group`. Los valores son fuerzas nodales (en las unidades del modelo, típicamente Newtons):

```
point_loads_by_node:
- {node_id: 3, uy: -15000.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}
```

## 3.6 Solver

```
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Los solvers disponibles y sus parámetros se documentan en el Capítulo 6.

### 3.6.1 Bloque convergence

Los solvers no lineales (`NonlinearSolver`, `ArcLengthSolver`) aceptan un bloque `convergence` que configura el criterio de parada de las iteraciones de Newton. La política sigue el patrón estándar de tolerancias mixtas absoluta + relativa, separado por dos criterios físicos: *fuerza* (residuo de equilibrio) y *desplazamiento* (incremento del iterado). La forma exacta de la comparación es:

```
||R[libres]|| <= atol_force + rtol_force * max(||F_ext||, ||F_int||)
||dU|| <= atol_disp + rtol_disp * ||U||
```

Ambos criterios deben cumplirse **simultáneamente** (semántica AND) para dar el paso por convergido.

**Parámetros disponibles** (todos opcionales, con defaults razonables):

- `rtol_force` (default `1.0e-5`): tolerancia relativa del residuo de fuerza. Es el parámetro que normalmente ajustarás: bajarlo a `1.0e-7` para análisis de precisión, subirlo a `1.0e-3` para exploración rápida.
- `rtol_disp` (default `1.0e-5`): tolerancia relativa de la corrección de desplazamiento. En problemas casi-rígidos puede afinarse independientemente del de fuerza.
- `atol_force_factor` (default `1.0e-9`): factor adimensional que define el piso absoluto de fuerza. La tolerancia absoluta efectiva se autoderiva como `atol_force_factor · escala_inicial`, donde la escala se calcula en el primer ensamblaje. Solo conviene ajustarlo si el análisis tiene tramos donde la carga característica colapsa transitoriamente y aparece falsa no-convergencia.
- `atol_disp_factor` (default `1.0e-9`): análogo para desplazamiento.

**Por qué los atol se autoderivan.** El término absoluto vive en las unidades del problema (newtons para fuerza, metros para desplazamiento). Codificarlo como constante global obligaría a ajustarlo al cambiar de unidades (N/m, kN/mm, MPa/mm...). En su lugar, Fenix calcula la escala característica del problema en el primer ensamblaje y multiplica por el factor adimensional. El resultado: **el mismo análisis planteado en distintos sistemas de unidades converge en el mismo número de iteraciones hasta paridad de bits**, sin tocar las tolerancias. La justificación arquitectural está en el ADR 0007.

**Cuándo afinar.** Para la inmensa mayoría de análisis los defaults son suficientes. Considera ajustar:

- `rtol_force` y `rtol_disp` más estrictos cuando publiques resultados o compares contra solución analítica (`1.0e-7` a `1.0e-9`).
- `rtol_disp` más laxo que `rtol_force` en problemas casi-rígidos donde el incremento de desplazamiento es naturalmente pequeño y baja muy rápido, dejando que el criterio de fuerza domine.
- Los factores `atol_*` prácticamente nunca; solo si encuentras un análisis específico donde la escala colapsa y los logs muestran iteraciones oscilando cerca del umbral.

**Nota sobre el solver algebraico interno.** El sistema lineal  $\mathbf{K} \delta \mathbf{U} = \mathbf{R}$  que aparece dentro de cada iteración se resuelve con un backend algebraico (Cholesky, LU...) seleccionado *automáticamente* según las propiedades de  $\mathbf{K}$ . Esta elección **no** es decisión de modelado y no requiere configuración. Si por motivos de diagnóstico necesitas forzar un backend específico, consulta el capítulo *Anexos técnicos — Capa algebraica* del Manual de Referencia.

## 3.7 Salida: output

Configura qué se exporta y cuándo:

```
output:
  file_name: "resultados_placa"
  pre_process: {export: true}      # exporta el modelo sin deformar (paso
    0)
  results:
    frequency: "all_steps"         # "last_step" o "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State", "Sigma_XX"]
  text_export:                    # opcional: salida estilo FEAP en .txt
    export: true
    nodes: all
    elements: all
```

### Nota

Cuando `frequency: "all_steps"` y el solver es no lineal, se genera un archivo `.vtu` por paso convergido más un archivo maestro `.pvd` que ParaView interpreta como animación.

# Capítulo 4

## Catálogo de Elementos Finitos

El motor expone dos familias de elementos: **1D** (estructuras reticulares: armaduras, cables, marcos) y **2D** (continuo plano: cuadrilátero y triángulo). Todos se referencian desde el YAML por su nombre exacto en `type`:

### 4.1 Elementos 1D

#### 4.1.1 Armaduras: `Truss2D` / `Truss2DCorot` / `Truss3D` / `Truss3DCorot`

Barra biarticulada que transmite exclusivamente esfuerzo axial. Cuatro variantes según dimensión y régimen geométrico:

| Elemento     | DOFs/nodo  | Dimensión | Régimen geom. | Uso                                         |
|--------------|------------|-----------|---------------|---------------------------------------------|
| Truss2D      | ux, uy     | 2D        | lineal        | cargas pequeñas, rotaciones pequeñas        |
| Truss2DCorot | ux, uy     | 2D        | corotacional  | grandes desplazamientos y rotaciones planas |
| Truss3D      | ux, uy, uz | 3D        | lineal        | armaduras espaciales en régimen lineal      |
| Truss3DCorot | ux, uy, uz | 3D        | corotacional  | grandes rotaciones en el espacio            |

Tabla 4.1: Familia de armaduras.

**Parámetros:** A (área de la sección transversal). Convención de signos:  $\sigma > 0 \Leftrightarrow \varepsilon > 0$  (elongación).

**Régimen de validez:**  $|\varepsilon| \lesssim 10^{-2}$ . Las variantes `Corot` aceptan rotaciones de cualquier magnitud entre commits; las versiones lineales asumen rotaciones pequeñas.

```
elements:
- id: 1
  type: Truss2DCorot
  material: 1
```

```
nodes: [1, 2]
A: 0.0005
```

### 4.1.2 Cables: Cable2DCorot / Cable3DCorot

Elemento 1D que transmite *únicamente* esfuerzo de tensión (no resiste compresión). La unilateralidad la aporta el material (típicamente `CableMaterial1D`, ver Capítulo 5); el elemento implementa la cinemática corotacional y la transferencia de la deformación al material.

- **DOFs/nodo:**  $u_x, u_y$  (2D);  $u_x, u_y, u_z$  (3D).
- **Parámetros:**  $A$ .
- **Cinemática:** corotacional (longitud y cosenos directores se recalculan en cada evaluación).
- **Régimen de validez:**  $|\varepsilon| \lesssim 10^{-2}$  tensado. En estado destensado el elemento aporta rigidez nula al sistema global; otros elementos deben garantizar la estabilidad numérica.

#### Advertencia

Un cable completamente destensado ( $\sigma = 0$ ) tiene  $\mathbf{K}_T = \mathbf{0}$  en sus DOFs. Si todos los caminos de carga del modelo dependen del cable y este se destensa, la matriz tangente global se vuelve singular. Pretensar el cable o garantizar redundancia estructural.

```
materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9

elements:
- id: 1
  type: Cable2DCorot
  material: 1
  nodes: [1, 2]
  A: 5.0e-5
```

### 4.1.3 Marcos 2D: Frame2DEuler / Frame2DTimoshenko / Frame2DEulerCorot

Elementos viga 2D que transmiten axial, cortante y momento flector. Tres variantes:

- **Frame2DEuler:** vigas esbeltas ( $L/h \gtrsim 10$ ), Euler-Bernoulli, régimen geométricamente lineal. Parámetros:  $A, I$ .
- **Frame2DTimoshenko:** vigas peraltadas o cortas ( $L/h \lesssim 10$ ). Incluye deformación por cortante; corrige automáticamente *shear locking* en el límite esbelto. Parámetros:  $A, I, A_s$  (área efectiva de cortante),  $\nu$  (opcional si el material expone  $\nu$ ).



- **Frame2DEulerCorot**: variante corotacional de Euler-Bernoulli. Captura grandes desplazamientos y grandes rotaciones rígidas del elemento (rotaciones deformacionales nodales moderadas,  $|\bar{\theta}| \lesssim 30^\circ$ ). Parámetros: **A**, **I**.

DOFs/nodo: **ux**, **uy**, **rz**. Convención:  $\sigma > 0 \Leftrightarrow \varepsilon > 0$  (elongación) en el eje axial;  $r_z$  positivo antihorario.

#### Advertencia

**Plasticidad por flexión.** Los elementos marco pasan al material únicamente la deformación axial centroidal  $(u_j - u_i)/L$ ; el módulo tangente  $E_t$  devuelto escala toda la matriz local (axial y flexión por igual). Esto significa que los marcos reproducen correctamente *plasticidad axial pura* pero *no* forman rótulas plásticas por flexión: la fluencia por momento requiere integración de  $\sigma(y)$  sobre la sección, característica que se incorporará en una clase **FiberSection** futura.

```
elements:
- id: 1
  type: Frame2DEulerCorot
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833
```

#### 4.1.4 Marco 3D: Frame3D

Viga 3D Euler-Bernoulli con 6 DOFs/nodo (**ux**, **uy**, **uz**, **rx**, **ry**, **rz**). Transmite axial, cortantes en dos planos, flexiones en dos planos y torsión de Saint-Venant pura. Régimen geoméricamente lineal (la variante corotacional 3D queda como pendiente).

**Parámetros:**

- **A** — área de la sección.
- **Iy**, **Iz** — momentos de inercia respecto a los ejes locales  $y$ ,  $z$ .
- **J** — constante torsional de Saint-Venant.
- **nu** — coeficiente de Poisson (opcional; se toma del material si lo expone).
- **ref\_vector** — vector de referencia (opcional, default  $[0,0,1]$ ) que fija la orientación de los ejes locales  $y$ ,  $z$  de la sección. Para barras casi-verticales se sugiere fijarlo explícitamente.

```

elements:
- id: 1
  type: Frame3D
  material: 1
  nodes: [1, 2]
  A: 0.01
  Iy: 8.33e-6
  Iz: 8.33e-6
  J: 1.66e-5
  ref_vector: [0.0, 0.0, 1.0]

```

## 4.2 Elementos 2D

La importación de mallas desde Gmsh instancia automáticamente los elementos 2D según la topología detectada (cuadriláteros  $\rightarrow$  Quad4; triángulos  $\rightarrow$  Tri3). También pueden declararse manualmente desde el bloque `elements`.

### 4.2.1 Cuadrilátero Bilineal: Quad4

Elemento isoparamétrico de 4 nodos, integración Gauss configurable.

- **DOFs/nodo:**  $u_x, u_y$  (STRAIN\_DIM = 3, Voigt  $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ ).
- **Nodos:** 4, ordenados en sentido antihorario.
- **Cuadratura:** "2x2" por defecto (4 puntos). "1x1" disponible pero produce modos espurios (*hourglassing*).
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Implementación:** kernels críticos compilados con `@njit` (Numba).
- **Limitación:** bloqueo volumétrico con materiales casi-incompresibles ( $\nu \rightarrow 0,5$ ); en ese régimen requeriría formulación mixta (no implementada).

### 4.2.2 Triángulo de Deformación Constante: Tri3

Triángulo CST de 3 nodos, un único punto de integración.

- **DOFs/nodo:**  $u_x, u_y$  (STRAIN\_DIM = 3).
- **Cuadratura:** 1 punto (deformación uniforme).
- **Parámetros:** `thickness`.
- **Limitación:** *shear locking* severo, convergencia lenta. **Preferir Quad4** salvo en transiciones donde Quad4 no encaja geométricamente.

# Capítulo 5

## Catálogo de Modelos Constitutivos

### 5.1 Elasticidad Lineal

#### 5.1.1 Elastic1D — elasticidad lineal axial

Ley de Hooke escalar  $\sigma = E \cdot \varepsilon$ . Sin variables internas. Compatible con todos los elementos 1D (armaduras y marcos).

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
```

#### 5.1.2 Elastic2D — elasticidad lineal isótropa 2D

Tensor constitutivo isótropo  $\sigma = \mathbf{C} \cdot \varepsilon$  en notación Voigt  $[xx, yy, xy]$ . Sin variables internas. Compatible con Quad4, Tri3.

```
materials:
- id: 1
  type: Elastic2D
  E: 200.0e9
  nu: 0.3
  hypothesis: plane_stress
```

La hipótesis `plane_stress` se usa para placas delgadas sin confinamiento normal; `plane_strain` para secciones de presa, túneles o problemas con extensión infinita en el eje longitudinal.

### 5.2 Plasticidad

#### 5.2.1 Elastoplastic1D — plasticidad J2 axial con endurecimiento isótropo

Plasticidad asociativa con criterio  $f = |\sigma_{\text{trial}}| - (\sigma_y + H\alpha) \leq 0$ . Algoritmo de *return mapping* clásico 1D; tangente algorítmica consistente  $E_t = E H / (E + H)$ .

- **Parámetros:**  $E$ ,  $\sigma_y$  (fluencia inicial),  $H$  (módulo de endurecimiento;  $H=0$  = perfectamente plástico).
- **Variables internas:**  $\varepsilon_p$  (deformación plástica),  $\alpha$  (acumulada equivalente).
- **Compatible con:** armaduras y marcos 1D (en marcos solo se aplica al esfuerzo axial; ver advertencia en el Capítulo 4).

```
materials:
- id: 1
  type: Elastoplastic1D
  E: 200.0e9
  sigma_y: 250.0e6
  H: 2.0e9
```

### 5.2.2 VonMises2D — plasticidad J2 plana con endurecimiento isótropo

Return mapping radial sobre la parte desviadora. Tangente algorítmica consistente.

- **Parámetros:**  $E$ ,  $\nu$ ,  $\sigma_y$ ,  $H$ , **hypothesis** (obligatorio `plane_strain`).
- **Variables internas:**  $\varepsilon_p$  tensorial (4 componentes  $[xx, yy, zz, xy]$ ),  $\alpha$ .
- **Implementación:** `kernel _compute_j2_plasticity` con `@njit`.

#### Advertencia

**Restricción cinemática:** solo `plane_strain`. Asignar `plane_stress` a `VonMises2D` lanza `NotImplementedError` (el return mapping en estado plano de esfuerzos requiere algoritmo distinto, no implementado).

```
materials:
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_strain
```

## 5.3 Daño Mecánico

### 5.3.1 IsotropicDamage1D — daño isótropo 1D con softening exponencial

Daño escalar  $d \in [0, 1)$  con módulo secante  $E_{\text{sec}} = (1 - d) E$ . Evolución por norma de la deformación equivalente  $\varepsilon_{eq} = |\varepsilon|$  y umbral  $\kappa_0$ :

$$d = 1 - \frac{\kappa_0}{\kappa} \exp(-\alpha(\kappa - \kappa_0)) \quad \text{para } \kappa > \kappa_0$$

- **Parámetros:** E, kappa\_0 (umbral elástico), alpha (velocidad de softening).
- **Variables internas:**  $\kappa$  (máxima histórica),  $d$  (daño).
- **Tangente:** secante (no algorítmica consistente; adecuado para Newton amortiguado, no óptimo para arc-length agresivo).

### 5.3.2 IsotropicDamage2D — daño isótropo 2D

Extensión 2D del modelo anterior. Tensor constitutivo secante  $\mathbf{C}_{\text{sec}} = (1 - d) \mathbf{C}_e$ .

- **Deformación equivalente:**  $\varepsilon_{eq} = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2}$ .
- **Parámetros:** E, nu, kappa\_0, alpha, hypothesis.
- **Limitación:**  $\varepsilon_{eq}$  simétrica no distingue daño en tensión vs compresión; para hormigón se requiere modelo de Mazars o split tensión/compresión (no implementado).

```
materials:
- id: 3
  type: IsotropicDamage2D
  E: 30.0e9
  nu: 0.2
  kappa_0: 1.5e-4
  alpha: 800.0
  hypothesis: plane_stress
```

## 5.4 Elasticidad Unilateral (Cable)

### 5.4.1 CableMaterial1D — cable lineal en tensión, nulo en compresión

Material 1D *memoryless* que captura la propiedad esencial del cable: solo transmite esfuerzo cuando está tensado.

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E \varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

con  $\langle \cdot \rangle^+$  el operador de MacAuley (parte positiva). Tangente discontinua en  $\varepsilon = 0$  (asignación al régimen compresivo por convención).

- **Parámetros:** E (módulo de Young en tensión).
- **Sin variables internas** (la respuesta depende sólo de  $\varepsilon$  instantánea).

- **Caveats numéricos:** en transiciones tensado  $\leftrightarrow$  destensado Newton-Raphson puede oscilar. Mitigación: usar `ArcLengthSolver` o pasos finos con `adaptive`.

```
materials:  
- id: 1  
  type: CableMaterial1D  
  E: 150.0e9
```

# Capítulo 6

## Esquemas de Solución

### 6.1 LinearSolver — solución directa lineal

Resuelve  $\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$  en un único `spsolve`. Las condiciones de Dirichlet se imponen por eliminación directa (ADR 0004): el sistema entregado al backend algebraico es ya el reducido  $\mathbf{K}_{\text{red}} \cdot \mathbf{U}_{\text{libre}} = \mathbf{F}_{\text{red}}$ .

- **Cuándo usarlo:** problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **Parámetros:** `linear_algebra` (default `auto`; ADR 0003 §4).
- **No aplica:** cualquier no-linealidad material (daño, plasticidad, cable) o geométrica (corotacional). Producirá resultados inconsistentes.

```
solver:  
  type: LinearSolver
```

### 6.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

Bucle externo de incrementos del factor de carga  $\lambda \in [0, 1]$ ; bucle interno Newton-Raphson sobre  $\mathbf{K}_t \cdot \Delta \mathbf{U} = \mathbf{R} = \lambda \mathbf{F}_{\text{ext}} - \mathbf{F}_{\text{int}}$ .

**Criterio de convergencia dual:**

$$\text{máx}(\text{err}_{\text{disp}}, \text{err}_{\text{force}}) < \text{tol}$$

con  $\text{err}_{\text{disp}} = \|\Delta \mathbf{U}\| / (\|\mathbf{U}\| + \epsilon)$  y  $\text{err}_{\text{force}} = \|\mathbf{R}\| / \text{máx}(\|\lambda \mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|, \epsilon)$ .

**Adaptatividad** (`adaptive: true`):

- Si converge en menos de 5 iteraciones, agranda el siguiente paso ( $\times 1.5$ ).
- Si no converge, biseca el paso ( $\div 2$ ). Falla definitivamente si  $\Delta \lambda < \text{min\_delta\_lambda}$ .

```

solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

**Parámetros:** convergence (bloque con rtol\_force, rtol\_disp, atol\_force\_factor, atol\_disp\_factor; ver ADR 0007), max\_iter, num\_steps, adaptive, min\_delta\_lambda, linear\_algebra, freeze\_tangent\_after\_iter.

**Limitación:** no puede atravesar puntos límite (snap-through) ni recorrer ramas con derivada negativa de la curva carga-desplazamiento. Para esos casos → ArcLengthSolver.

### 6.3 ArcLengthSolver — longitud de arco cilíndrico (Crisfield)

Traza curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga  $\lambda$ .

**Esquema:**

- **Predictor tangente:**  $\Delta \mathbf{U}_t = \mathbf{K}_t^{-1} \cdot \mathbf{F}_{\text{ext}}^{\text{ref}}$ ;  $\Delta \lambda = \pm dl / \|\Delta \mathbf{U}_t\|$ . Signo elegido por proyección con el incremento previo.
- **Corrector iterativo:** en cada iteración resuelve dos sistemas ( $\mathbf{K} \cdot \Delta \mathbf{U}_R = \mathbf{R}$  y  $\mathbf{K} \cdot \Delta \mathbf{U}_t = \mathbf{F}_{\text{ext}}$ ) y aplica la restricción cuadrática cilíndrica  $\|\delta \mathbf{U}\|^2 = dl^2$ .
- **Auto-ajuste de dl:** se agranda en convergencias rápidas, se reduce en lentas, biseca al fallar.
- **Caso especial final\_step:** si el predictor sobrepasaría max\_lambda, fija  $\lambda$  a ese valor y resuelve solo desplazamientos (Newton-Raphson puro).

```

solver:
  type: ArcLengthSolver
  max_iter: 15
  max_lambda: 1.0
  initial_dl: 0.05
  max_steps: 200
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

**Cuándo usarlo:** problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando NonlinearSolver diverge cerca de un punto límite. Más caro por paso (dos spsolve por iteración) pero indispensable en estos casos.

**Referencia:** Crisfield, “A fast incremental/iterative solution procedure that handles snap-through” (*Computers & Structures*, 1981).



# Capítulo 7

## Post-Procesamiento y Visualización

### 7.1 Formato VTU (VTK XML)

Mediante `meshio`, Fenix FEM exporta los resultados en archivos `.vtu` compatibles con ParaView. La información se clasifica en dos diccionarios estándar:

#### 7.1.1 Datos Nodales (*Point Data*)

- **Displacements:** campo vectorial  $(u_x, u_y, u_z)$  por nodo.
- **External\_Forces:** distribución nodal del vector de cargas aplicado en el paso actual.
- **Supports:** indicador booleano de DOFs restringidos.

#### 7.1.2 Datos de Elemento (*Cell Data*)

- **Von\_Mises:** tensión equivalente J2 (cuando aplica).
- **Internal\_State:** variable principal del material según su `PRIMARY_STATE_VAR` (acumulada equivalente  $\alpha$  en plasticidad,  $d$  en daño, etc.).
- **Sigma\_XX, Sigma\_YY, Tau\_XY:** componentes del tensor de esfuerzos en notación Voigt.

### 7.2 Animación con Archivo PVD

Cuando se exporta con `frequency: .all_steps`", además de un `.vtu` por paso se genera un archivo maestro `.pvd` (XML colección) que ParaView abre como animación temporal. El “tiempo” en el archivo PVD coincide con el factor de carga  $\lambda$ .

### 7.3 Salida en Texto Plano (Estilo FEAP)

Opcional para verificación manual o post-procesamiento con scripts. Configurable por nodos y elementos seleccionados:

```
output:
  text_export:
    export: true
    nodes: [1, 5, 10]          # o "all"
    elements: all
    nodal_results: ["Displacements", "External_Forces"]
    element_results: ["Von_Mises", "Internal_State"]
```

## 7.4 Workflow ParaView

1. File >Open y seleccionar el archivo .pvd (o el grupo de .vtu).
2. *Apply* en el panel *Properties*.
3. Reproducir la animación con los controles temporales de la barra superior.
4. Aplicar filtro **Warp By Vector** con **Displacements** para visualizar la deformación física (ajustar **Scale Factor** para amplificar).
5. Cambiar el coloreado de **Solid Color** a la variable de interés (**Von\_Mises**, **Internal\_State**, **Sigma\_XX**, ...).

# Capítulo 8

## Ejemplos Completos

### 8.1 Marco 2D — Viga en Voladizo

Combinación de un tramo Euler-Bernoulli y un tramo Timoshenko, empotrada en el extremo y cargada en la punta.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}

materials:
- id: 1
  type: Elastic1D
  E: 200.0e9

elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

point_loads_by_node:
- {node_id: 3, uy: -15000.0}

solver:
```

```
type: LinearSolver

output:
  file_name: "viga_voladizo_marco"
  pre_process: {export: true}
  results:
    frequency: "last_step"
    nodal_results: ["Displacements"]
```

Listing 8.1: Marco 2D mixto Euler/Timoshenko

## 8.2 Placa Continua con Plasticidad J2 (Gmsh)

Placa con perforación central importada desde `.msh`, sometida a carga axial creciente hasta superar el límite elástico del acero. Solver no lineal con paso adaptativo.

```
mesh: "placa_agujero.msh"

mesh_physical_groups:
  "Superficie_Placa":
    material: 1
    thickness: 0.05
    quadrature: "2x2"

materials:
  - id: 1
    type: VonMises2D
    E: 200.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 1.5e9
    hypothesis: plane_strain

boundary_conditions_by_group:
  "Borde_Fijo": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}

solver:
  type: NonlinearSolver
  num_steps: 30
  max_iter: 10
  tol: 1.0e-5
  adaptive: true

output:
  file_name: "estudio_placa"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
```

```
element_results: ["Von_Mises", "Internal_State"]
```

Listing 8.2: Placa perforada elastoplástica

**Interpretación:** durante los primeros pasos (régimen elástico) Newton-Raphson converge en 1–2 iteraciones. Cuando la concentración de esfuerzos en la periferia del orificio supera  $\sigma_y = 250$  MPa, comienzan iteraciones del *return mapping*; **Internal\_State** (la deformación plástica acumulada  $\alpha$ ) crece visiblemente alrededor del agujero.

## 8.3 Cable Pretensado en Régimen Corotacional

Cable 2D con material unilateral, cargado transversalmente en su nodo central. Demuestra el acoplamiento elemento corotacional + material unilateral.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [5.0, 0.0]}
- {id: 3, coords: [10.0, 0.0]}

materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9

elements:
- id: 1
  type: Cable2DCorot
  material: 1
  nodes: [1, 2]
  A: 5.0e-5

- id: 2
  type: Cable2DCorot
  material: 1
  nodes: [2, 3]
  A: 5.0e-5

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0}
- {node_id: 3, ux: 0.0, uy: 0.0}

point_loads_by_node:
- {node_id: 2, uy: -500.0}

solver:
  type: NonlinearSolver
  num_steps: 50
  max_iter: 25
  tol: 1.0e-5
  adaptive: true

output:
```

```
file_name: "cable_carga_transversal"  
pre_process: {export: true}  
results:  
  frequency: "all_steps"  
  nodal_results: ["Displacements"]
```

Listing 8.3: Cable bajo carga transversal

# Capítulo 9

## Uso Avanzado: API Programática

Aunque el flujo principal es *data-driven* (YAML), el motor se puede usar como librería Python para escenarios donde el archivo no es suficiente: optimización paramétrica, generación procedural de mallas, acoplamiento con bibliotecas externas, scripting de batería de tests.

### 9.1 Patrón Fachada

Los componentes principales se exponen desde la raíz del paquete:

```
import numpy as np
from fenix import Domain, VonMises2D, Quad4, Assembler, NonlinearSolver,
    VtkExporter

# 1. Dominio y nodos
domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [1.0, 0.0])
n3 = domain.add_node(3, [1.0, 1.0])
n4 = domain.add_node(4, [0.0, 1.0])

# 2. Material y elemento
acero = VonMises2D(E=200e9, nu=0.3, sigma_y=250e6, H=2e9, hypothesis='
    plane_strain')
placa = Quad4(element_id=1, nodes=[n1, n2, n3, n4], material=acero,
    thickness=0.1)
domain.add_element(placa)

# 3. Restricciones (Dirichlet)
n1.fix_dof('ux', 0.0); n1.fix_dof('uy', 0.0)
n4.fix_dof('ux', 0.0); n4.fix_dof('uy', 0.0)

domain.generate_equation_numbers()

# 4. Cargas (Neumann)
F_ext = np.zeros(domain.total_dofs)
F_ext[n2.dofs['ux']] = 150000.0
F_ext[n3.dofs['ux']] = 150000.0
```

```
# 5. Resolución
assembler = Assembler(domain)
solver = NonlinearSolver(assembler, num_steps=10, adaptive=True)
U = solver.solve(F_ext)

# 6. Exportación
VtkExporter(domain).export("resultados.vtu", U=U, F_ext=F_ext)
```

Listing 9.1: Script estructural directo



# Capítulo 10

## Diagnóstico de Problemas

- ***YamlValidationError: parámetro ‘X’ no aceptado por ‘Y’.***  
El parser detecta un parámetro que el constructor del elemento no admite. La salida lista los parámetros aceptados; revisar el catálogo o la spec del componente.
- ***Matriz singular detectada.***  
Restricciones cinemáticas insuficientes: el dominio puede trasladarse o rotar como cuerpo rígido. Verificar que al menos un nodo tiene fijos los suficientes DOFs para anclar el dominio. En modelos con cables o materiales con softening pronunciado, comprobar que existen caminos de carga alternativos al elemento que se destensa o degrada.
- ***Newton-Raphson no convergió / bisecando incremento.***  
El paso de carga es demasiado grande. Aumentar `num_steps`, asegurar `adaptive: true`. Verificar tipeo de los parámetros constitutivos ( $\sigma_y$ ,  $\kappa_0$ , etc.). Si el problema tiene snap-through o softening, cambiar a `ArcLengthSolver`.
- ***No se importó ningún elemento (Solid2D).***  
El parser de Gmsh no encontró superficies bidimensionales. En Gmsh, crear una *Plane Surface* y generar la malla 2D antes de exportar.
- ***NotImplementedError: VonMises2D requires plane\_strain.***  
El modelo `VonMises2D` solo soporta deformación plana. Cambiar `hypothesis` a `plane_strain` en el material o reemplazar por un modelo compatible con plane stress (no implementado actualmente).
- ***Cable totalmente destensado —  $K_T = 0$ .***  
Pretensar el cable o garantizar que la estructura tiene rigidez residual por otros elementos. Considerar también pasos de carga finos para capturar la transición tensado  $\rightarrow$  destensado.

# Índice Alfabético

Gmsh, [2](#)

Instalación, [1](#)

ParaView, [2](#)

YAML, [5](#)